

Apache Kafka & Event-Driven Systems — Complete Study Guide

For a Senior Engineer • anchored to the Ford SCA-V connected-vehicle telemetry platform

Why this guide exists. Kafka has its own dialect. Generic messaging/JMS words — "publisher," "subscriber," "queue," "message id" — are *not* Kafka's vocabulary, and using them in an interview signals you've worked around messaging rather than inside Kafka. This guide teaches the concepts *and* drills the exact terms, with your real 12-broker telemetry platform as the running example.

The vocabulary map (read this first, memorize it last)

Generic / JMS term	Kafka's actual term
-----	-----
Publisher	Producer
Subscriber	Consumer
Queue	Topic (+ Consumer Group for load-sharing)
(topic subdivision)	Partition
Message ID	Offset
Copies for durability ..	Replication / Replicas
Primary / backup	Leader / Follower
Server	Broker
Group of servers	Cluster

Everything below explains *why* each term means what it does.

1. Topic

Definition. A topic is a named, append-only log that messages are published to and read from — the central channel of Kafka.

How it works. Producers write messages to a topic; consumers read from it. Unlike a traditional queue, a topic is a *durable log* — messages aren't deleted when read; they're retained for a configured time (e.g. 7 days), so many independent consumers can read the same topic at their own pace.

Ford example. The SCA-V platform has a topic like `vehicle-telemetry` that every connected vehicle's data flows into. Separate topics exist for different event types — `vehicle-telemetry`,

`vehicle-diagnostics`, `vehicle-alerts` — so consumers subscribe to exactly what they need.

ASCII sketch — a topic is a log:

```
Topic: vehicle-telemetry (append-only log)
+----+----+----+----+----+----+----+
| m0 | m1 | m2 | m3 | m4 | m5 | m6 | <- new messages appended to the end -->
+----+----+----+----+----+----+----+
      ^                               ^
    oldest (retained N days) newest

Reading does NOT delete. Consumers track their own position.
```

Interview line. "A topic is a durable, append-only log — not a queue. Messages are retained by time or size, so multiple consumers can independently read the same data."

2. Partition

Definition. A partition is one slice of a topic; a topic is split into multiple partitions to allow parallelism and horizontal scale.

How it works. Each partition is an ordered, independent log living on a broker. Ordering is guaranteed *within* a partition but **not across** partitions. The number of partitions sets the maximum consumer parallelism — one partition can be read by at most one consumer *in a group* at a time. A message's partition is chosen by its **key** (same key → same partition → ordering preserved for that key).

Ford example. `vehicle-telemetry` is split into, say, 24 partitions, keyed by **VIN** (vehicle ID). All events from one vehicle always land in the same partition, so that vehicle's event order is preserved — critical for reconstructing a trip timeline — while 24 partitions let 24 consumers process different vehicles in parallel.

ASCII sketch — topic split into partitions, keyed by VIN:

```
Topic: vehicle-telemetry
Partition 0: [VIN-A m0][VIN-A m1][VIN-A m2]      (all VIN-A events, in order)
Partition 1: [VIN-B m0][VIN-B m1]
Partition 2: [VIN-C m0][VIN-C m1][VIN-C m2]
...
key = VIN -> hash(VIN) % numPartitions -> same VIN always same partition

Ordering: GUARANTEED within a partition, NOT across partitions.
```

Interview line. "Partitions are how Kafka scales and parallelizes a topic. Ordering is only guaranteed within a partition, so I key by VIN to keep each vehicle's events ordered while still parallelizing across vehicles."

3. Producer

Definition. A producer is the application/client that publishes (writes) messages to a Kafka topic. (Kafka term — *not* "publisher.")

How it works. The producer chooses the topic and, via the message key, influences the partition. It batches messages for throughput and waits for acknowledgments (`acks`) based on the durability it needs. It doesn't know or care who consumes the data.

Ford example. An edge ingestion service acts as the producer — it receives raw frames from vehicles, serializes them (Avro), and writes them to `vehicle-telemetry` keyed by VIN, at millions of events per day across the fleet.

ASCII sketch:

```
Edge Ingestion (PRODUCER)
  | key=VIN, value=Avro(telemetry)
  v
[ Kafka: vehicle-telemetry ] -> routed to partition by hash(VIN)
```

Interview line. "The producer writes to a topic and controls the key, which determines the partition. In Kafka we say producer, not publisher."

4. Consumer

Definition. A consumer is the application/client that reads (subscribes to and processes) messages from a topic. (Kafka term — *not* "subscriber.")

How it works. A consumer reads from one or more partitions, tracking its position via the **offset**. It periodically **commits** its offset so that after a restart it resumes where it left off, not from the beginning. Consumers pull data (they request it) rather than having it pushed.

Ford example. A trip-reconstruction service consumes `vehicle-telemetry`, reading each vehicle's ordered events to rebuild journeys; a separate anomaly-detection service consumes the *same* topic independently for real-time alerts — both read the same log without interfering, because each tracks its own offsets.

Interview line. "A consumer reads from partitions and tracks its offset so it can resume after a restart. Multiple consumers can read the same topic independently because each manages its own position."

5. Consumer Group

Definition. A consumer group is a set of consumers that cooperate to share the work of reading a topic, with each partition assigned to exactly one consumer in the group.

How it works. Kafka distributes the topic's partitions across the consumers in a group — this is how you scale out consumption. If a topic has 24 partitions and the group has 6 consumers, each reads 4 partitions. Add consumers (up to the partition count) to scale; if one dies, Kafka **rebalances** its partitions to the survivors. Different *groups* each get the full stream independently (that's how you fan out to multiple services).

Ford example. The anomaly-detection service runs as a consumer group of 8 instances across the 24-partition topic — each instance handles 3 partitions, and if one instance crashes, its partitions rebalance to the other 7 with no data loss. Meanwhile the trip-reconstruction service is a *separate* group reading the same topic in full.

ASCII sketch — one group shares partitions; different groups each get everything:

```
Topic vehicle-telemetry: partitions P0..P5

Consumer Group "anomaly-detection":
  consumer-1 -> P0, P1
  consumer-2 -> P2, P3
  consumer-3 -> P4, P5      (load shared; each partition to ONE consumer)

Consumer Group "trip-reconstruction": (independent - gets the FULL stream)
  consumer-A -> P0, P1, P2, P3, P4, P5

Rule: within a group, 1 partition -> 1 consumer.
      across groups, everyone gets all the data.
```

Interview line. "A consumer group shares a topic's partitions across its members for scale — one partition per consumer within the group. Different groups each get the whole stream, which is how you fan out to multiple independent services."

6. Offset

Definition. An offset is the unique, sequential position/ID of a message within a partition.

How it works. Each message in a partition gets a monotonically increasing offset (0, 1, 2, ...). Consumers track and **commit** their offset to record how far they've read. This is *not* a correlation ID (an app-level tracing concept) — it's Kafka's built-in position marker. Committing offsets is what makes consumption resumable and is central to delivery guarantees.

Ford example. If the anomaly-detection service is at offset 10,432 on partition 3 and restarts, it resumes from 10,433 — no reprocessing, no gaps — because it committed its offset.

ASCII sketch:

```
Partition 3:  offset:  0    1    2    3    4    5    6
                  [m0] [m1] [m2] [m3] [m4] [m5] [m6]
                        ^
                  consumer committed offset = 3 (next read = 4)
Restart -> resume at offset 4. No replay, no loss.
```

Interview line. "The offset is the message's sequential position in a partition. Consumers commit offsets to track progress and resume after a restart — it's Kafka's position marker, not an application correlation ID."

7. Replication, Replicas, Leader & Follower

Definition. Replication copies each partition across multiple brokers for fault tolerance; the copies are replicas — one is the leader (handles all reads/writes), the rest are followers (stay in sync).

How it works. A **replication factor** of 3 means each partition exists on 3 brokers. Producers and consumers talk only to the **leader**; **followers** replicate the leader's data. If the leader's broker dies, Kafka promotes an in-sync follower to leader automatically — no data loss, no manual intervention. The set of followers caught up with the leader are the **ISR (in-sync replicas)**.

Ford example. The 12-broker cluster runs `vehicle-telemetry` with replication factor 3 and `min.insync.replicas=2`. When a broker went down during a data-center event, the affected partitions' followers were promoted to leaders and telemetry ingestion continued uninterrupted — the durability guarantee that made the platform trustworthy for connected-vehicle data.

ASCII sketch — one partition, replication factor 3:

Partition 0, replication factor = 3

Broker 1	Broker 2	Broker 3
P0 = LEADER	P0 = follower	P0 = follower
(reads/writes)	(replicates)	(replicates)

^ producers/consumers talk ONLY to the leader

leader broker dies -> an in-sync follower is promoted to leader -> no data loss

Interview line. "Replication copies each partition across brokers. Clients talk to the leader; followers stay in sync as replicas. If the leader dies, an in-sync replica is promoted automatically. With replication factor 3 and min.insync.replicas 2, we tolerate a broker failure with no data loss."

8. Broker & Cluster

Definition. A broker is a single Kafka server; a cluster is the group of brokers working together.

How it works. Each broker hosts some partitions (leaders and followers) and handles client requests for them. The cluster coordinates partition assignment, leader election, and replication. (Historically coordination used ZooKeeper; newer Kafka uses **KRaft**, a built-in Raft-based controller, removing the ZooKeeper dependency.)

Ford example. "12-broker cluster" means 12 Kafka servers sharing the partitions of all topics, spread across availability zones so the loss of one broker — or one zone — doesn't stop ingestion.

Interview line. "A broker is one Kafka server; the cluster is the set of brokers. They coordinate partition placement and leader election — via KRaft in current versions, ZooKeeper in older ones."

9. Producer acks (delivery durability)

Definition. `acks` is the producer setting controlling how many broker acknowledgments to wait for before considering a write successful: `0`, `1`, or `all`.

How it works.

- `acks=0` — fire-and-forget; fastest, can lose data (no confirmation).
- `acks=1` — leader confirms; loses data only if the leader dies before followers replicate.

- `acks=all` (or `-1`) — leader *and* all in-sync replicas confirm; strongest durability, needed for exactly-once. Pair with `min.insync.replicas`.

Ford example. Telemetry that must not be lost uses `acks=all` with `min.insync.replicas=2`, trading a little latency for the guarantee that a committed vehicle event survives a broker failure.

ASCII sketch:

```

acks=0    producer --msg--> leader          (no wait)          fastest, least safe
acks=1    producer --msg--> leader --ack->          safe unless leader
dies
acks=all  producer --msg--> leader + ISR followers --ack->  safest (exactly-
once)

```

Interview line. "acks controls durability: 0 is fire-and-forget, 1 waits for the leader, all waits for the leader plus in-sync replicas. For data we can't lose I use `acks=all` with `min.insync.replicas 2`."

10. Delivery semantics: at-most-once, at-least-once, exactly-once

Definition. The three guarantees for how many times a message can be delivered/processed.

How it works.

- **At-most-once** — commit offset *before* processing; if you crash mid-process, the message is lost but never duplicated. (Rarely wanted.)
- **At-least-once** — process *then* commit; if you crash before committing, you reprocess — so no loss, but possible **duplicates**. This is the common default, and it's why consumers must be **idempotent**.
- **Exactly-once (EOS)** — no loss and no duplicates, achieved with Kafka transactions and idempotent producers. More overhead; use where correctness demands it.

Ford example. Telemetry ingestion runs at-least-once with idempotent consumers (dedupe by VIN + event timestamp), so a rebalance-driven reprocess never creates a duplicate trip event. Where exactly-once was required (billing-grade counts), Kafka transactions were used.

ASCII sketch — the duplicate/loss trade-off:

```
at-most-once :  commit THEN process  -> crash = message LOST      (no dupes)
at-least-once:  process THEN commit   -> crash = REPROCESS        (possible
dupes)
exactly-once :  transactional         -> no loss AND no dupes     (more
overhead)

at-least-once is the common default -> make consumers IDEMPOTENT.
```

Interview line. "At-least-once is the usual default — you process then commit, so a crash means reprocessing, which is why consumers must be idempotent. Exactly-once uses Kafka transactions and idempotent producers when duplicates are unacceptable."

11. Schema Registry (with Avro)

Definition. The Schema Registry is a separate service that stores and versions message schemas (commonly Avro) and validates that producers and consumers agree on the data format.

How it works. Instead of embedding the full schema in every message, the producer registers the schema and sends a small schema ID with each Avro-serialized message; consumers fetch the schema by ID to deserialize. The registry enforces **compatibility rules** (e.g. backward-compatible) so a producer can't push a breaking schema change that would break existing consumers.

Ford example. SCA-V used the Confluent Schema Registry with Avro for `vehicle-telemetry`. When the telemetry payload gained a new field, backward-compatibility rules let producers evolve the schema without breaking the running consumers — essential when you can't upgrade every service simultaneously across a large platform.

Interview line. "The Schema Registry stores and versions Avro schemas and enforces compatibility, so producers and consumers stay in sync and schemas can evolve without breaking consumers. Messages carry a schema ID, not the full schema."

12. Dead Letter Topic / Queue (DLT / DLQ)

Definition. A dead-letter topic is where messages are routed when a consumer can't process them successfully after retries.

How it works. When a message repeatedly fails (bad data, downstream unavailable), rather than blocking the partition or dropping it silently, the consumer routes it to a separate DLT for

later inspection, alerting, and replay. This keeps the main stream flowing while preserving the failed message.

Ford example. Malformed telemetry frames that failed Avro deserialization were routed to a `vehicle-telemetry-dlt`, with CloudWatch/monitoring alerts, so the ops team could inspect and replay them without stalling live ingestion.

ASCII sketch:

```
main topic --> consumer --process--> ok
                |
                | fails after N retries
                v
            dead-letter topic (vehicle-telemetry-dlt) --> alert + manual replay
```

Interview line. "A dead-letter topic captures messages that fail processing after retries, so the main stream keeps flowing and failures can be inspected and replayed instead of lost or blocking the partition."

13. Event-driven architecture (the bigger picture)

Definition. An event-driven architecture is one where services communicate by producing and reacting to events (facts that happened) rather than calling each other directly.

How it works. Instead of Service A synchronously calling Service B (tight coupling), A publishes an event to Kafka and any interested service consumes it independently. This **decouples** producers from consumers — you can add a new consumer without touching the producer — and absorbs load spikes, since Kafka buffers events the consumers process at their own pace.

Choreography vs. orchestration. In **choreography**, services react to each other's events with no central coordinator (simple, but hard to follow as it grows). In **orchestration**, a central coordinator drives a multi-step workflow (clearer for complex flows with compensation — the Saga pattern).

Ford example. A vehicle event published once to Kafka is consumed independently by trip-reconstruction, anomaly-detection, and analytics — three teams building on the same stream without coordinating deployments. Adding a fourth consumer (say, a new dashboard) required zero changes to the producer.

ASCII sketch — decoupling via events:

TIGHTLY COUPLED (direct calls):

Ingestion -> Trip svc -> Anomaly svc -> Analytics (chain; one break stops all)

EVENT-DRIVEN (via Kafka):

Ingestion --event--> [Kafka topic] --+--> Trip reconstruction
+--> Anomaly detection
+--> Analytics

add a new consumer any time; producer never changes

Interview line. "Event-driven means services publish and react to events instead of calling each other directly. That decouples them — I can add a consumer without changing the producer — and Kafka buffers load spikes. For multi-step workflows with compensation I'd use orchestration (a Saga); for simple reactions, choreography."

One-page cheat sheet (drill this until automatic)

Topic named append-only log; messages retained, not deleted on read

Partition slice of a topic; ordering only WITHIN a partition; keyed routing

Producer writes to a topic (NOT "publisher")

Consumer reads from a topic (NOT "subscriber")

Consumer Group consumers sharing partitions; 1 partition -> 1 consumer in-group

Offset message's sequential position in a partition (NOT correlation id)

Replication copies partitions across brokers for durability

Replica a copy of a partition

Leader the replica serving reads/writes

Follower replicas that copy the leader (ISR = in-sync replicas)

Broker one Kafka server

Cluster the group of brokers (coordinated by KRaft, or ZooKeeper in old versions)

acks producer durability: 0 / 1 / all

at-least-once process then commit -> possible dupes -> need idempotency

exactly-once transactions + idempotent producer -> no loss, no dupes

Schema Registry stores/versions Avro schemas; enforces compatibility

DLT / DLQ dead-letter topic for messages that fail after retries

Event-driven publish/react to events; decouples services; Kafka buffers spikes

The five interview questions you'll most likely get (with the crisp answer)

1. **"Why partitions?"** → Parallelism and scale. Ordering is guaranteed only within a partition, so I key by a business ID (VIN) to keep related events ordered while parallelizing across keys.
2. **"How does Kafka guarantee no data loss?"** → Replication (factor 3), `acks=all`, and `min.insync.replicas=2`. Clients talk to the leader; if it dies, an in-sync replica is promoted. A committed message survives a broker failure.
3. **"At-least-once vs exactly-once?"** → At-least-once (process then commit) is the common default and can produce duplicates, so consumers must be idempotent. Exactly-once uses Kafka transactions plus an idempotent producer when duplicates are unacceptable.
4. **"How do consumer groups scale?"** → Partitions are shared across the group, one partition per consumer. Add consumers up to the partition count to scale; a dead consumer's partitions rebalance to the survivors. Different groups each get the full stream.
5. **"Kafka vs RabbitMQ?"** → Kafka is a durable, replayable log built for high-throughput streaming and multiple independent consumers; RabbitMQ is a traditional broker/queue optimized for routing and per-message delivery/ack semantics. Use Kafka for event streaming and replay, RabbitMQ for complex routing and task queues.

Study note: the terms that scrambled in rapid-fire — producer (not publisher), consumer (not subscriber), topic/partition (not queue/replicas), offset (not correlation id), leader/follower — are exactly the ones interviewers use to tell hands-on Kafka experience from generic messaging familiarity. You have the hands-on experience; drill the one-page cheat sheet until the Kafka word comes out before the generic one.

Version note: Kafka is moving from ZooKeeper-based coordination to KRaft; exact defaults and some APIs vary by version. Verify version-specific details against the Apache Kafka docs for your cluster's version.